

一.线程有关的概念

线程：从进程的资源中分割一部分出来用于线程的执行，线程需要依赖于进程而存在(进程退出了，线程也没了)
进程和线程都是为了实现多任务的并发执行

主线程和子线程

主线程：你调用pthread_create函数来创建子线程，调用该函数的那个进程我们叫做主线程

进程是操作系统分配资源的最小单位：操作系统在分配资源(内存资源)的时候以进程为单位来分配的，每个进程都拥有独立的内存资源

线程是操作系统调度的最小单位：cpu执行进程/线程都是在不断的切换

二.线程相关的接口函数

1.线程的创建

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void *), void *arg);
```

参数：thread --》长整型，线程的ID号

attr --》结构体类型，线程的属性，后面会单独讨论，先不理睬

void *(*start_routine) (void *) 重点重点--》函数指针，线程的任务函数，你创建线程需要做什么事情就靠它来实现

arg --》传递给start_routine的参数

传参数给线程有两种常见的做法

方法1：可以通过pthread_create第四个参数来搞定

方法2：可以通过全局变量(跟进程的区别)

Compile and link with -pthread //编译程序的时候需要链接线程库,具体格式如下所示

gcc 线程的创建.c -pthread

或者 gcc 线程的创建.c -lpthread

2.线程的退出与回收

```
void pthread_exit(void *retval)
```

参数：retval --》存放线程的退出信息

注意这个地址一定不能返回局部变量的地址，因为局部变量的地址会被释放

```
int pthread_join(pthread_t thread, void **retval) //阻塞主函数，等待线程的退出，回收线程
```

参数：thread --》你要回收的线程ID

retval --》保存线程退出时候返回的信息

3.线程的取消 --》强行把线程干掉

```
int pthread_cancel(pthread_t thread);
```

参数：thread --》线程的ID

4.设置线程的取消状态

```
int pthread_setcancelstate(int state, int *oldstate);
```

参数：state --》PTHREAD_CANCEL_ENABLE 线程可以被取消(默认情况下，所有的线程都能被取消)

PTHREAD_CANCEL_DISABLE 线程不可以被取消

oldstate --》备份的，保存线程原本的取消状态

5.设置线程的取消类型

```
int pthread_setcanceltype(int type, int *oldtype);
```

参数: type --》PTHREAD_CANCEL_DEFERRED 延时取消(线程默认就是延时取消)

当你调用pthread_cancel取消线程的时候, 延时取消表示线程会继续往后运行直到遇到

取消点函数才退出

取消点函数: man 7 pthreads 在第七本手册里面查询线程的有关介绍

验证延迟取消的情况。

```
xxxxx; <---- 取消请求
```

```
xxxxx;
```

```
xxxxx;
```

```
fputc()
```

... 6.线程延时取消,会运行到取消点.c 1.19KB

... 7.线程立即取消,不会运行到取消点.c 1.29KB

PTHREAD_CANCEL_ASYNCHRONOUS 立即取消(调用pthread_cancel函数的时候线程立

马退出)

oldtype --》保存线程原本的取消类型

三.练习作业

1.创建多个线程, 每个线程执行不同的任务

2.传递一个学生结构体变量给线程

```
struct student
{
    char name[20];
    int age;
};
```

传递一个字符串给线程

3.用多线程实现

线程1: 死循环读取判断触摸屏坐标

线程2: 循环显示bmp图片